INF384 · TEMAS AVANZADOS EN INGENIERÍA DE SOFTWARE 1 · 2026-2

Sesión 05

Despliegue continuo: artefactos e infraestructura como código

Promoción de un artefacto único entre ambientes y definición de la infraestructura como código versionado

Capítulo 4 (1 de 2) · 3 horas | RA2 | Prof. José Felipe del Castillo Salazar

PUCP

INF384 · 9 CAPÍTULOS · 14 SESIONES · 42 HORAS

La ruta del curso



SESIÓN 5

Objetivo de la sesión y ruta

OBJETIVO

Diseñar el mecanismo de promoción de un artefacto entre ambientes e implementar la infraestructura del ambiente como código (IaC), **justificando qué garantía aporta cada decisión.**

1. **Entrega continua y despliegue continuo**
   ¿Cuándo elegir cada una?
2. **Gestión de artefactos**
   Registro, inmutabilidad y promoción del mismo binario entre ambientes.
3. **Infraestructura como código**
   Estado declarado, plan como control previo y desviación respecto de lo declarado.
4. **Las Fijas del Lab 3**

Resultado de aprendizaje vinculado: RA2 — diseñar e implementar pipelines de integración y despliegue continuo.

CONTINUIDAD CON LA SESIÓN 4

Dónde estamos en el pipeline

Commit	Build	Pruebas	Calidad	Seguridad	Publicar	Desplegar

Lo que ya está resuelto

- El artefacto se construye una sola vez, desde el archivo de bloqueo.
- Calidad y seguridad detienen la publicación cuando el umbral no se cumple.
- El artefacto se publica versionado y trazable al commit que lo originó.
- El pipeline responde en minutos, no en horas.

Lo que ese pipeline no responde

- ¿Dónde queda ese artefacto y quién lo va a buscar para desplegarlo?
- ¿Cómo llega el mismo binario a QA y a producción sin volver a construirse?
- ¿Quién creó el ambiente que lo recibe, y se puede volver a crear igual?
- ¿Qué pasa cuando alguien cambia ese ambiente?

El artefacto validado todavía no es un sistema en funcionamiento. Falta el ambiente donde corre y el mecanismo que lo lleva hasta ahí.

A

Gestión de artefactos

Registro, inmutabilidad y promoción del mismo binario entre ambientes

Entrega continua y despliegue continuo

Entrega continua

Cada cambio que pasa el pipeline queda listo para desplegarse a producción en cualquier momento. La publicación ocurre cuando una persona lo decide.

La capacidad técnica existe; el disparador es humano.

Despliegue continuo

Cada cambio que pasa el pipeline llega a producción sin intervención manual. No hay botón: la validación automática es la única autorización.

Exige todo lo de entrega continua, más confianza en la validación.

1

La diferencia es una decisión de gobierno y madurez, no de herramienta

El mismo pipeline sirve para las dos. Lo que cambia es si existe una compuerta manual antes de producción.

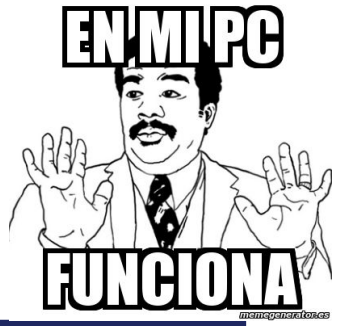
2

La mayoría de organizaciones opera en entrega continua

Y está bien: el despliegue continuo sin observabilidad ni reversión automática traslada el costo del error al usuario.

Usar la sigla CD sin especificar cuál de las dos es la fuente de confusión más frecuente al describir la madurez de un equipo, normalmente es entrega continua (Continuous Delivery)

Qué es exactamente un artefacto desplegable



Unidad versionada e inmutable que contiene todo lo necesario para ejecutar el sistema, salvo la configuración propia del ambiente (variables de entorno).

Paquete de aplicación

Código ya construido y sus dependencias resueltas, comprimido en un solo archivo.
Ejemplos:
EAR, JAR, WAR, Bundle

Imagen de contenedor

Aplicación, dependencias y sistema base en un solo objeto. Lo que corre en QA y en producción es idéntico hasta el sistema operativo.

Plantilla de infraestructura

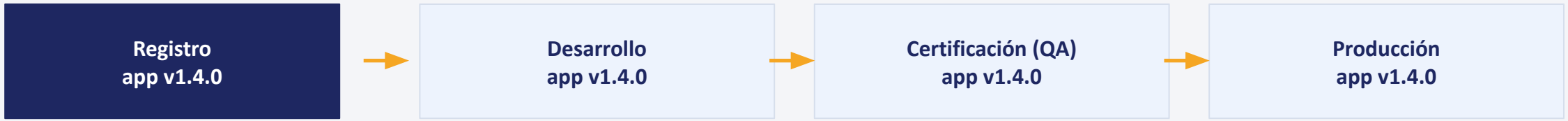
Definición versionada del ambiente. Se promueve y se aplica igual que el código.
Ejemplo: Terraform, Cloudformation, CDK, Bicep

1. **Versionado**
   Un identificador único que nadie reutiliza.
2. **Inmutable**
   Construido una vez, nunca editado después.
3. **Trazable**
   Registro del commit exacto que lo produjo.
4. **Autocontenido**
   Solo le falta la configuración del ambiente para ejeutarse.

Si el contenido del artefacto puede cambiar después de construido, ninguna de las validaciones de CI están vigentes.

El registro de artefactos y la promoción entre ambientes

UN SOLO ARTEFACTO · TRES AMBIENTES



1

Promover: mover artefacto al siguiente ambiente/estadío

El ambiente siguiente apunta al mismo objeto binario. Nada se vuelve a construir, nada se vuelve a resolver.

2

Cada ambiente aporta una evidencia distinta

El artefacto acumula validaciones a medida que avanza. Lo que llega a producción ya pasó todas las anteriores.

3

El registro guarda: qué y de dónde viene.

Versión, commit de origen, ejecución del pipeline que lo produjo y resultado de las validaciones.

Sin trazabilidad de lo que está desplegado en producción el soporte en producción es un CAOS

Tag y Huella Criptográfica

Referencia por etiqueta (tag)

La etiqueta es un puntero que alguien puede reasignar a otro contenido. Desplegar por etiqueta hoy y mañana puede entregar dos artefactos distintos, porque pude cambiar la referencia.

El ambiente queda descrito por un nombre que no identifica nada.

Referencia por huella criptográfica

Nace cuando se genera el artefacto en base al contenido. Identifica de forma exacta y única.

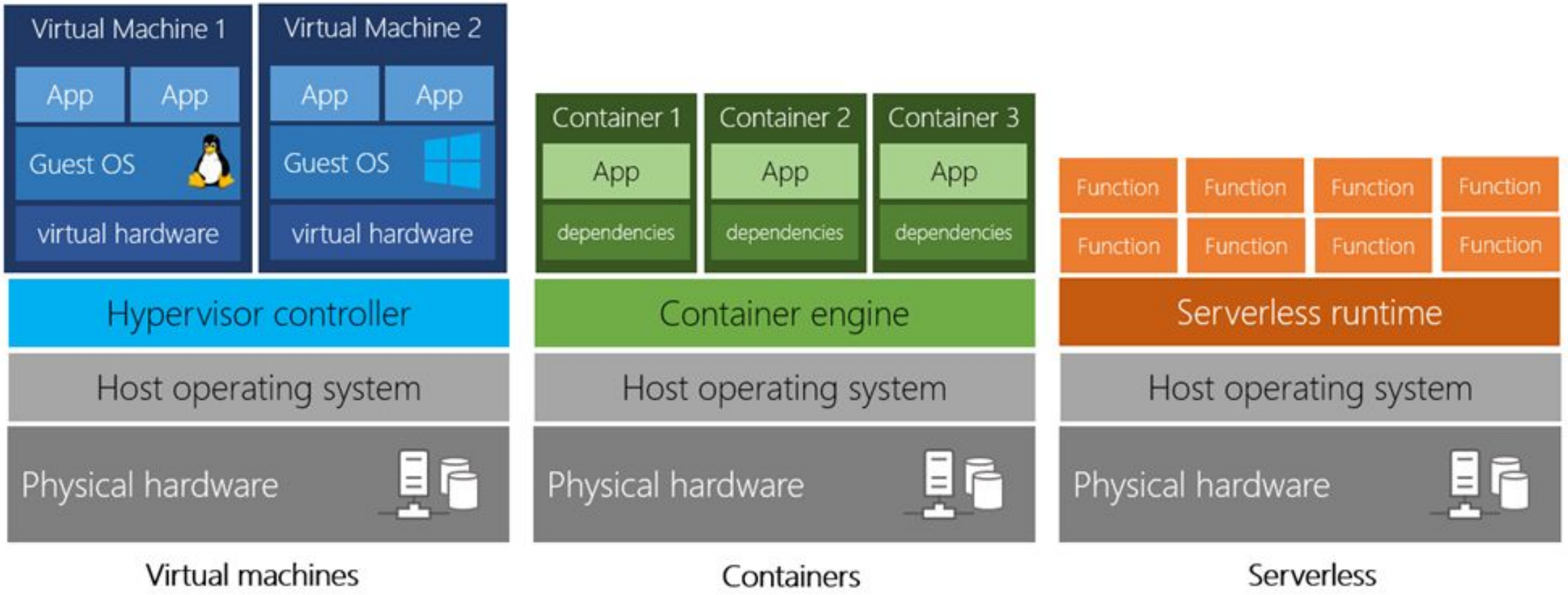
El ambiente queda descrito por el contenido que realmente corre.

LAS DOS FORMAS DE REFERIRSE AL MISMO ARTEFACTO

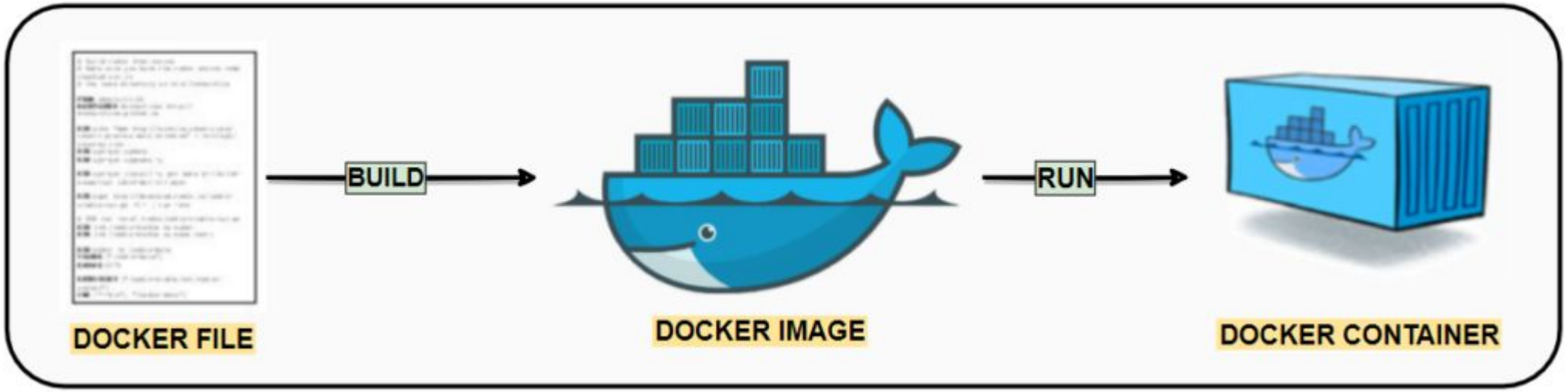
```
registro/servicio-despachos:latest              # puntero móvil: hoy y mañana pueden diferir
registro/servicio-despachos:1.4.0               # versión fija por convención del equipo, igual puede cambiar
registro/servicio-despachos@sha256:9f2c...a41b  # huella del contenido: identifica un solo artefacto
```

La respuesta a «qué está corriendo en producción» es la huella criptográfica

VM / Contenedores / Serverless



Docker File / Docker Image / Docker Container





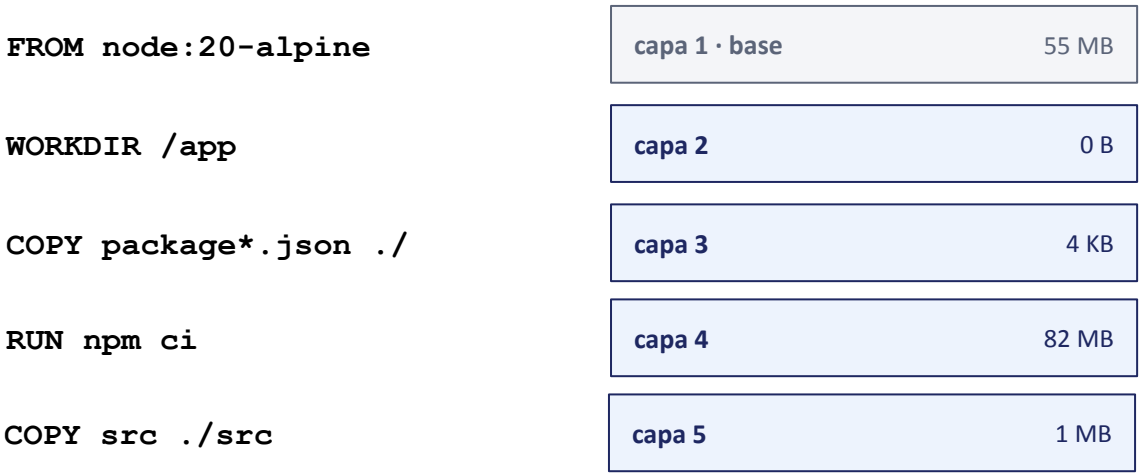




Cómo se construye y se almacena una imagen

Una imagen es una pila ordenada de capas de solo lectura, más un manifiesto que declara qué proceso se ejecuta al arrancar.

CADA INSTRUCCIÓN DEL DOCKERFILE PRODUCE UNA CAPA



1 La imagen es la pila de capas completa

Lo que una capa escribe queda guardado, y las siguientes trabajan encima.

2 Las capas se guardan por contenido

Si el contenido no cambió, la build toma esa capa de la caché.

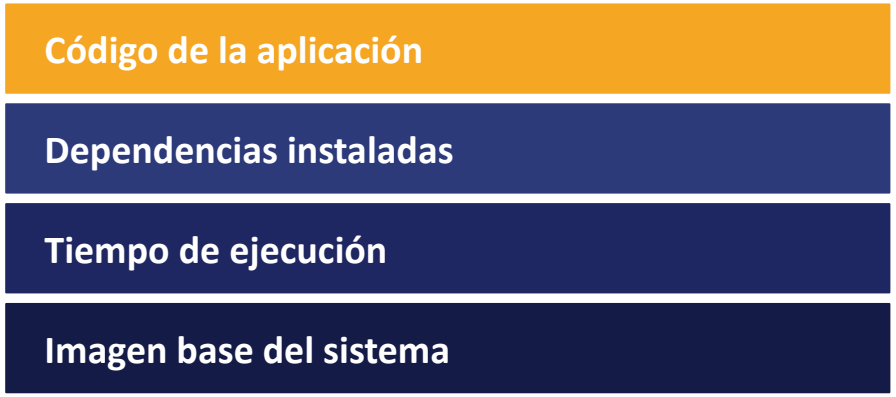
3 Un cambio invalida su capa y las de arriba

El orden de las instrucciones decide cuánto se reconstruye en cada build.

4 El contenedor agrega una capa escribible

Lo que el proceso escriba vive ahí y desaparece al terminar.

Anatomía de una imagen de contenedor



Cada instrucción del archivo de construcción produce una capa. Las capas de abajo cambian poco y se reutilizan entre construcciones.

1

Declarar la imagen base es una decisión de Seguridad

Todo lo que trae la base viaja dentro del artefacto y hereda sus vulnerabilidades conocidas.

2

Construcción en varias etapas

Permite ir agregando lo que corresponde a cada etapa, partiendo de lo que dejó la etapa anterior.

3

Tamaño: superficie de ataque y también tiempo

Menos componentes significa menos hallazgos que gestionar y despliegues más rápidos.

4

Necesitamos declarar el usuario del contenedor

Por defecto corre como root, lo cuál tiene mayor potencial de vulnerabilidades.

El Capítulo 6 analiza la imagen como objeto a escanear. Hoy importa como artefacto único que se promueve entre ambientes.

Un Dockerfile con malas prácticas

SERVICIO DE DESPACHOS · DOCKERFILE

```
FROM node:latest                        # 1

WORKDIR /app

COPY . .                                # 2


RUN npm install                         # 3


ENV DB_PASSWORD=Pucp2026                # 4


RUN apt-get update && apt-get install -y \
    curl vim git build-essential                # 5

EXPOSE 3000

                                    # 6

CMD npm start
```

Un Dockerfile con malas prácticas

SERVICIO DE DESPACHOS · DOCKERFILE

```
FROM node:latest                        # 1

WORKDIR /app

COPY . .                                # 2


RUN npm install                         # 3


ENV DB_PASSWORD=Pucp2026                # 4


RUN apt-get update && apt-get install -y \
    curl vim git build-essential                # 5

EXPOSE 3000

#No se especifica usuario               # 6

CMD npm start
```

1. **Tag móvil en la imagen base y una imagen MUY pesada**
   Dos builds del mismo commit pueden producir imágenes distintas.
2. **Copia de todo el contexto**
   Entran .git y .env, y la caché se invalida en cada cambio.
3. **Instalación que ignora el lock file**
   Resuelve versiones nuevas en cada build
4. **Secreto escrito en una capa**
   El valor queda en la imagen y se lee con docker history.
5. **Herramientas de compilación en la imagen final**
   curl, vim y git viajan a producción y amplían la superficie de ataque.
6. **El proceso corre como root**
   Una vulnerabilidad de la aplicación se ejecuta con el usuario más privilegiado.

Todo lo que este archivo declara queda dentro de la imagen y viaja a cada ambiente donde se despliegue.

Un Docker File con Best Practices

SERVICIO DE DESPACHOS · DOCKERFILE CORREGIDO

```
FROM node:20.11-alpine AS build          # 1

WORKDIR /app

COPY package.json package-lock.json ./   # 2

RUN npm ci                               # 3

COPY src ./src
RUN npm run build && npm prune --omit=dev

FROM node:20.11-alpine AS runtime        # 4
ENV NODE_ENV=production
WORKDIR /app

COPY --from=build /app/dist ./dist       # 5
COPY --from=build /app/node_modules ./node_modules

USER node                                # 6

EXPOSE 3000
HEALTHCHECK --interval=30s \
  CMD node dist/healthcheck.js
ENTRYPOINT ["node", "dist/server.js"]    # 7
```

1. **Versión fija de la imagen base y lo más ligera posible**
   El mismo commit produce la misma imagen en cualquier máquina.
2. **Manifiesto y lock file antes del código**
   La capa de dependencias se reutiliza mientras el lock file no cambie.
3. **Instalación desde el lock file**
   Instala las versiones del lock file y falla si está desactualizado.
4. **Dos etapas: build y runtime**
   Las herramientas de compilación se quedan en la etapa build.
5. **Solo el artefacto construido pasa a runtime**
   La imagen final contiene lo que se ejecuta en producción.
6. **Usuario sin privilegios declarado**
   El proceso corre con una identidad acotada dentro del contenedor.
7. **Healthcheck y forma exec**
   El orquestador conoce el estado del proceso y lo detiene limpio.

Un archivo .dockerignore con .git, node_modules y .env acompaña a este Dockerfile: reduce el contexto de build y mantiene fuera de la imagen el material del repositorio.

La configuración es lo único que cambia por ambiente

Dentro del artefacto

- Código construido
- Dependencias resueltas
- Valores por defecto del sistema
- Esquema de la configuración esperada

Inyectado en el despliegue

- URL de servicios y datos e conexión a bases de datos
- Tamaños, réplicas y límites de recursos
- Nivel de LOG y Feature Flags
- Identificador del ambiente

Fuera del repositorio y del artefacto

- Contraseñas y llaves de acceso
- Tokens de servicios externos
- Certificados privados
- Cualquier valor que rotar sea obligatorio

1 **El artefacto no sabe en qué ambiente está corriendo**

2 **El secreto se resuelve en el momento del despliegue**

Un valor distinto por ambiente dentro del artefacto obliga a una construcción por ambiente, y con eso se pierde la promoción.

Del build al registro

GITHUB ACTIONS — CONSTRUCCIÓN Y PUBLICACIÓN DEL ARTEFACTO

```
jobs:
  publicar:
    needs: [calidad, seguridad]          # 1
    if: github.ref == 'refs/heads/main'
    permissions:
      id-token: write                    # 2
      contents: read
    steps:
      - uses: actions/checkout@v4
      - name: Calcular version
        run: echo "VER=${{ VERSION }}"
             >> $GITHUB_ENV              # 3
      - name: Autenticar contra el registro
        uses: <accion-de-autenticacion>
      - name: Construir imagen
        run: docker build
             -t $REG/app:$VER
             -t $REG/app:$TAG_ENV
             -t $REG/app:${{ github.sha }} .    # 4
      - name: Publicar imagen
        run: docker push --all-tags $REG/app
```

1

Publica solo lo validado

El trabajo depende de las compuertas del Capítulo 3. Sin ellas aprobadas, no llega a ejecutarse.

2

Identidad federada, no llaves

El pipeline pide un token de corta duración en lugar de guardar credenciales permanentes.

3

La versión se hereda del BUILD

La versión se hereda del cálculo ya generado en la etapa de build.

4

Referencias al mismo objeto

La versión para las personas (VER y TAG_ENV) y la referencia del commit para la trazabilidad automática.

B

Infraestructura como código

El ambiente que recibe el artefacto, declarado, versionado y reconstruible

¿Qué problema resuelve IaC?

1. **El ambiente se creó a mano y nadie registró cómo**
   Reconstruirlo depende de que la persona que lo hizo recuerde el orden exacto de los pasos y los valores que puso.

2. **Certificación y producción difieren en algo pero nadie sabe en qué**
   El defecto que aparece solo en producción no está en el código: está en la diferencia que nadie documentó.

3. **Levantar un ambiente nuevo toma semanas y solo algunos saben como hacerlo**
   Aparece en el value stream map como espera, y convierte cualquier prueba realista en un privilegio que se pide por ticket.

4. **El cambio de infraestructura no pasa por ninguna revisión**
   Una modificación en la consola no tiene autor, ni revisor, ni posibilidad de revertirse. En el código, todo eso viene incluido.

Declarar la infraestructura como código permite todo lo que vimos en CI: versionamiento, revisión, validación automática y trazabilidad.

Infraestructura como Código

Enfoque imperativo

Se describen los pasos: crear la red, luego el balanceador, luego el servicio. Ejecutarlo dos veces produce dos ambientes o un error.

El autor es responsable de conocer el estado previo.

Herramientas: AWS CDK, CLIs, Pulumi

Enfoque declarativo

Se describe el resultado esperado: una red, un balanceador y un servicio con estas características. La herramienta calcula qué falta.

Ejecutarlo dos veces produce el mismo ambiente.

Herramientas: Terraform, Cloudformation, Bicep (Azure Resource Manager)

1

El estado es el registro de lo que la herramienta creó

Guarda la correspondencia entre cada recurso declarado y el recurso real. Sin él, la herramienta no sabe qué le pertenece.

2

El estado se guarda remoto y con bloqueo

En el equipo de una persona funciona en el disco local. Con dos personas aplicando a la vez, el estado local se corrompe.

3

El estado contiene valores sensibles

Guarda atributos de los recursos creados, y entre ellos puede haber credenciales generadas. Se protege como un secreto.

El archivo de estado es el activo más crítico de la infraestructura, debe quedar en un almacenamiento seguro y persistente (NUNCA en LOCAL)

Familias de herramientas y criterios de selección

Criterio	Nativa del proveedor	Multiproveedor	Lenguaje general	Orquestación
Forma de expresarlo	Plantilla declarativa	Lenguaje propio declarativo	Código de propósito general	Manifiesto declarativo
Estado	Lo administra el proveedor	Archivo propio a proteger	Archivo propio a proteger	Lo administra el clúster
Curva de entrada	Baja si ya se conoce la nube	Media	Baja para quien programa	Media
Portabilidad	Nula fuera del proveedor	Alta	Baja	Alta entre clústeres
Riesgo característico	Depender de un solo proveedor	Pérdida o bloqueo del estado	Abstracciones difíciles de auditar	Configuration Drift
Herramientas	Cloudformation	Terraform	PULUMI	YAML (Kubernetes)

La elección se justifica por el criterio que más pesa en el contexto: cuántos proveedores hay que sostener, quién va a mantener el código y qué tan auditable tiene que ser el cambio.

El plan es el control; la aplicación es la consecuencia

Código declarado	Estado registrado	Infraestructura real	PLAN diferencia calculada	APPLY diferencia ejecutada

1

El plan se lee antes de aplicar, y por alguien más

Es el equivalente a la revisión de código: la diferencia propuesta se discute cuando todavía no ocurrió nada.

2

Lo que destruye importa más que lo que crea

Un plan que recrea una base de datos se ve casi igual que uno que cambia una etiqueta. La lectura no es opcional.

3

El plan se calcula en la propuesta de cambio y se aplica desde la rama principal

El mismo patrón del Capítulo 3: validar en la propuesta, ejecutar solo lo que ya fue revisado y aprobado.

4

Aplicar dos veces el mismo código no cambia nada

La segunda ejecución calcula una diferencia vacía. Esa propiedad es lo que hace repetible al despliegue.

Aplicar sin leer el plan convierte un cambio de infraestructura en una apuesta sobre lo que la herramienta decidió hacer.

Cuando el ambiente real deja de parecerse a lo declarado

Un cambio manual en la consola resuelve el incidente de hoy y deja el código describiendo un ambiente que ya no existe.

Cómo aparece

Un incidente urgente, permisos amplios y una consola abierta. Casi siempre con buena intención y bajo presión de tiempo.

Qué rompe

El siguiente plan propone revertir el cambio manual, porque el código sigue siendo la única fuente declarada.

Cómo se detecta

Una ejecución periódica que calcula el plan sin aplicarlo. Si la diferencia no es vacía, hay desviación.

Cómo se cierra

El cambio manual se traslada al código y se vuelve a aplicar. La corrección del incidente se conserva; la desviación no.

Prohibir el acceso a la consola no elimina la desviación: la detección periódica es lo que la vuelve visible antes del siguiente despliegue.

La infraestructura recorre el mismo pipeline

GITHUB ACTIONS — INFRAESTRUCTURA COMO CÓDIGO

```
on:
  pull_request: { paths: ['infra/**'] }
  push:         { branches: [main] }

jobs:
  validar:
    steps:
      - run: <validacion-de-sintaxis>      # 1
      - run: <analisis-de-politicas>       # 2
  plan:
    needs: validar
    steps:
      - run: <calculo-del-plan>
      - uses: <publicar-plan-en-el-pr>     # 3
  aplicar:
    needs: plan
    if: github.ref == 'refs/heads/main'
    environment: produccion                # 4
    steps:
      - run: <aplicar-el-plan-aprobado>
```

1. **La sintaxis se valida antes que nada**
   El control más rápido va primero. Es el mismo orden por costo de detección del Capítulo 3.

2. **Las reglas se evalúan sobre la declaración**
   Un almacenamiento público o un puerto abierto se detecta antes de que exista el recurso.

3. **El plan queda escrito en la propuesta de cambio**
   Quien revisa ve la diferencia exacta. Sin eso, la aprobación es sobre el código y no sobre su efecto.

4. **La aplicación pasa por un ambiente protegido**
   Es la compuerta de aprobación, y es el tema con el que abre la sesión 6.

La infraestructura recorre el mismo pipeline

GITHUB ACTIONS — INFRAESTRUCTURA COMO CÓDIGO

```
on:
  pull_request:
  push:
jobs:
  inicial: #1
    steps:
      - run: terraform init        #instala librerías
      - run: terraform validate    #valida sintaxis
  plan:
    needs: validar  #valida que "compila" con las vars del ambiente"
    steps:
      - run:
      |
            terraform plan -out=tfplan  # 2- Genera plan de cambios
            terraform show -json tfplan > tfplan.json #formatea archivo
            checkov -f tfplan.json  # 3- Valida seguridad
  aplicar:
    needs: plan
    if: github.ref == 'refs/heads/main'
    environment: produccion
    steps:
      - run: terraform apply     #5 Despliegue de infraestructura
```

1

La sintaxis se valida antes que nada

El control más rápido va primero, en terraform valida sintaxis y consistencia interna.

2

El plan queda escrito en la propuesta de cambio

Quien revisa ve la diferencia exacta. Sin eso, la aprobación es sobre el código y no sobre su efecto.

3

Análisis de seguridad

Si no pasa la validación de seguridad no se despliega

4

Si pasa todas las validaciones, se aplican los cambios

Usa las credenciales del pipeline para conectarse a la nube destino y desplegar.

Kahoot de cierre

Viene con puntos adicionales:
1er puesto: +1 punto en parcial
2do puesto: +0.5 en parcial
3er puesto: +0.5 en parcial

SESIÓN 5

Laboratorio 3 - Las fijas

OBJETIVO

1- Implementar validaciones de calidad y seguridad en aplicación, generar un contenedor a partir de una imagen docker y desplegarla si es que pasa las validaciones.
2- Implementar validaciones de sintaxis, integridad, seguridad, validar si el plan cumple con lo que queremos cambiar y si es así desplegar la infraestructura

- Github Actions: como depende un step de otro

- Docker: docker image, build, push, tag

- Terraform: init, validate, plan, deploy, destroy
- Bonus:
  - ¿Qué es un Docker Registry? (dockerhub, AWS Elastic Container Registry ECR)

Repositorios con ejemplos

- Terraform example: https://github.com/josedelcastillo/terraform-example

- Docker example: https://github.com/josedelcastillo/docker-example

CIERRE SESIÓN 5

Qué se llevan

1. **Entrega continua y despliegue continuo se diferencian por una compuerta**
   La capacidad técnica es la misma; lo que cambia es quién autoriza la llegada a producción.

2. **Promover el artefacto no debe necesitar cambiar contenido**
   El mismo objeto binario recorre todos los ambientes, identificado por su huella digital / digest

3. **La configuración vive fuera del artefacto**
   Un valor por ambiente adentro obliga a una reconstrucción por ambiente y destruye la promoción.

4. **El ambiente declarado (Iac) se puede revisar, versionar y reconstruir**
   Y el plan es el momento en que ese cambio se discute, antes de que ocurra.

5. **Cambios por fuera de IaC se pueden identificar**
   Una ejecución periódica que calcula el plan sin aplicarlo es el control más barato para estos drifts.